# DSAA3073: Theories in Data Science

## Week 6: AdaBoost Algorithm & Convergence

Concise Learning Sheet

Wei Wang · DSA, HKUST(GZ) · 2026-07-15

Explorative projection

# Introduction: The Boosting Challen

The term "boosting" was coined by Robert Schapire in his 1990 paper, which gave the first theoretical construction showing that weak learning implies strong learning.

🎯 **Key Takeaway**

**Conceptual Backbone: From Convex Optimization to Ensemble Learning**

This week connects three fundamental ideas:

1. **Convex optimization** (Week 1 foundations): Gradient descent on smooth loss functions
2. **Coordinate descent**: Greedy optimization in high-dimensional spaces
3. **Ensemble learning**: Combining weak learners through adaptive reweighting

The journey: We start with the weak learning problem, derive AdaBoost through optimization principles, prove exponential convergence via convexity, and explain generalization through margin theory.

In the late 1980s, Michael Kearns and Leslie Valiant posed a fundamental question in computational learning theory: **Is weak learning equivalent to strong learning?** In other words, if we can learn slightly better than random guessing, can we learn arbitrarily well?

This question was not merely theoretical curiosity. Many practical learning algorithms produce classifiers that are only moderately accurate. If we could systematically combine many such "weak" classifiers into a highly accurate "strong" classifier, we would have a powerful general-purpose learning technique.

The answer turned out to be a resounding **yes**, leading to one of the most successful families of machine learning algorithms: **boosting algorithms**. The most famous of these, **AdaBoost** (Adaptive Boosting), developed by Yoav Freund and Robert Schapire in 1997, has become a cornerstone of modern machine learning.

## What Makes Boosting Special?

Boosting algorithms have several remarkable properties:

1. **Theoretical guarantee:** They provably convert weak learners into strong learners
2. **Practical effectiveness:** They achieve state-of-the-art performance on many tasks
3. **Simplicity:** The core algorithm is surprisingly simple and elegant
4. **Versatility:** They work with any weak learning algorithm as a black box
5. **Margin maximization:** They implicitly maximize classification margins, improving generalization

**Connection**

Boosting provides a constructive proof that weak PAC learnability implies strong PAC learnability under the usual PAC assumptions.

# Learning Objectives

By the end of this week, you will be able to:

1. Understand how AdaBoost combines weak learners (51% accuracy) into strong classifiers (99%+)
2. Explain the complete AdaBoost algorithm with all notation (D_t, F(x), H(x), $\alpha$_t)
3. Derive the reweighting formula from exponential loss minimization
4. Prove exponential convergence of training error: error $\leq \exp(-2\gamma^2 T)$
5. Understand why exponential loss is chosen as the optimization objective

✓ **Checkpoint**

**Checkpoint: Foundations**

Before proceeding, ensure you understand:

- Convex functions and gradient descent (Week 1)
- PAC learning framework (Week 2-3)
- Margin-based generalization (Week 5-6)

**Self-check:** Can you explain why convex loss functions are easier to optimize than 0-1 loss?

# Roadmap

This week covers boosting theory through six key sections:

- **Section 1: The Weak Learning Challenge** - transforming 51% accuracy into 99%+
- **Section 2: AdaBoost Algorithm Overview** - complete algorithm with notation definitions
- **Section 3: Why Exponential Loss?** - motivating the choice of loss function
- **Section 4: Deriving AdaBoost** - where the reweighting formula comes from
- **Section 5: Training Error Convergence** - exponential convergence guarantees
- **Section 6: Coordinate Descent Perspective** - unifying algorithmic and optimization views

Each section follows a problem-driven arc: student confusion → natural approach → crisis → resolution.

# The Weak Learning Impossibility

**? Student Question**

**"How is 51% accuracy useful?"**

I have a classifier that achieves 51% accuracy—just barely better than random guessing (50%). This seems essentially useless. To build anything practical, don't I need at least 70-80% accuracy?

Why would anyone waste time with such a weak classifier?

## The Direct Approach: Build a Stronger Classifier

Your instinct is reasonable: "51% is too weak, I need a stronger learning algorithm."

**Let's try training more complex classifiers on our dataset:**

| Classifier | Training Acc. | Test Acc. | Issue |
|---|---|---|---|
| Decision stump | 51% | 51% | Barely better than random |
| Depth-3 tree | 62% | 54% | Slight overfitting |
| Depth-5 tree | 78% | 52% | Severe overfitting |
| Neural network | 85% | 53% | Still overfits |

**Result:** After trying multiple approaches, we're stuck around 52-54% test accuracy. Complex models overfit, and the weak learner (51% stump) is the best we can reliably achieve.

## The Shocking Alternative: Combine Weak Learners

What if instead of replacing the weak learner, we **combine multiple copies** intelligently?

**⚠ Crisis**

**The Weak Learning Paradox**

Running AdaBoost with T=100 decision stumps (each 51% accurate):

- **Training error: $< 0.01\%$** (essentially perfect)
- **Test error: 10%** (strong generalization)

From "barely better than random" to "highly accurate" using the exact same 51% weak learner!

### The Transformation in Action

| Rounds T | Training Error | Test Error | Improvement |
|---|---|---|---|

| | | | |
|---|---|---|---|
| T=1 | 49% | 49% | baseline |
| T=10 | 35% | 38% | 11pp |
| T=50 | 0.8% | 12% | 37pp |
| T=100 | 0.01% | 10% | 39pp |
| T=200 | $< 0.001\%$ | 9% | 40pp |

**The gap:** From 49% error (single stump) to 0.01% error (100 stumps)—using the same "useless" 51% base classifier!

💡 **Aha Moment**

**The Resolution: Complementary Specialists**

Each weak learner only needs 51% accuracy on **its specific weighted distribution**. Through adaptive reweighting:

1. **Round 1:** Learn easy patterns (51% on uniform distribution)
2. **Round 2:** Focus on what round 1 missed (51% on reweighted data)
3. **Round 3:** Focus on what rounds 1-2 missed (51% on new weighting)
4. **Combined:** Each learner covers different mistakes → strong ensemble

**Key insight:** Weak learners don't need individual strength—they need to be **complementary**!

## The Formal Guarantee

**Definition: Weak Learner**

A **weak learner** produces hypothesis $h : \mathcal{X} \to \{-1, +1\}$ with weighted error:

$$\varepsilon = \sum_{i=1}^{m} D(i) \cdot \mathbb{1}[h(x_i) \neq y_i] \leq \frac{1}{2} - \gamma \tag{1}$$

for some **edge** $\gamma > 0$ (independent of distribution $D$).

**Theorem: Training Error Convergence**

If each weak learner has edge $\gamma > 0$, then after $T$ rounds AdaBoost achieves:

$$\text{Training error} \leq \exp(-2\gamma^2 T) \tag{2}$$

This decreases **exponentially** with rounds.

**Key insight:** Even tiny edge $\gamma$=0.01 succeeds—just needs more rounds. The formula:

$$T \approx \frac{\ln\left(\frac{1}{\text{target error}}\right)}{2\gamma^2} \tag{3}$$

For $\gamma = 0.10$ (60% accuracy): just 230 rounds achieves 1% training error.

## Why This Works: The Complementarity Principle

**Key Takeaway**

**From Individual Weakness to Collective Strength**

1. **Each learner is weak individually** (51% accuracy)
2. **But focuses on different mistakes** (via reweighting)
3. **Ensemble covers all gaps** (complementary specialists)
4. **Exponential convergence** (convexity of loss function)

Result: Transform arbitrarily weak learners ($\gamma > 0$) into arbitrarily strong classifiers.

Next, we'll derive exactly **how** AdaBoost achieves this through its reweighting mechanism.

# Understanding the AdaBoost Algori

The notation $D_t(i)$ emphasizes that $D_t$ is a **probability distribution** over training examples. At each round, we sample from (or weight by) this distribution when training the weak learner.

Before we dive into **why** AdaBoost works, let's first understand **v** introduces the complete algorithm with all notation defined clearly, followed by a concrete numerical example that shows how the algorithm works in practice.

## Notation and Setup

**Key Takeaway**

**Essential Notation for AdaBoost**

- **Training set:** $S = \{(x_1, y_1), ..., (x_m, y_m)\}$ where $x_i \in \mathcal{X}$ and $y_i \in \{-1, +1\}$
- **Distribution weights:** $D_t(i)$ = weight on example $i$ at round $t$, where $\sum_{i=1}^{m} D_t(i) = 1$
- **Weak hypothesis:** $h_t : \mathcal{X} \to \{-1, +1\}$ trained on distribution $D_t$
- **Hypothesis weight:** $\alpha_t \in \mathbb{R}^+$ = importance of hypothesis $h_t$ in final ensemble
- **Ensemble model:** $F(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$ (weighted sum, **not** a classifier yet)
- **Final classifier:** $H(x) = \text{sign}(F(x))$ = final prediction after $T$ rounds

**Key insight:** $F(x)$ is a **real-valued** function (margin), not a classifier. The sign determines the predicted label, but the magnitude $|F(x)|$ measures confidence.

## The Complete AdaBoost Algorithm

Here is the full algorithm with all steps explicit:

**Algorithm: AdaBoost (Adaptive Boosting)**

**Input:**

- Training set $S = \{(x_1, y_1), ..., (x_m, y_m)\}$ with $y_i \in \{-1, +1\}$
- Weak learning algorithm WeakLearn
- Number of rounds $T$

**Initialize:** Set uniform distribution:

$$D_1(i) = \frac{1}{m} \quad \text{for all } i = 1, ..., m \tag{4}$$

**For** $t = 1, 2, ..., T$:

1. **Train weak learner:** Call WeakLearn with distribution $D_t$ to get hypothesis $h_t : \mathcal{X} \to \{-1, +1\}$
2. **Compute weighted error:**

$$\varepsilon_t = \sum_{i=1}^{m} D_t(i) \cdot \mathbb{1}[h_t(x_i) \neq y_i] = \mathbb{P}_{i \sim D_t}[h_t(x_i) \neq y_i] \tag{5}$$

3. **Compute hypothesis weight:** (requires $\varepsilon_t < 1/2$)

$$\alpha_t = \frac{1}{2} \ln \frac{1 - \varepsilon_t}{\varepsilon_t} \tag{6}$$

4. **Update distribution:**

$$D_{t+1}(i) = \frac{D_t(i) \cdot \exp(-\alpha_t y_i h_t(x_i))}{Z_t} \tag{7}$$

where normalization constant:

$$Z_t = \sum_{i=1}^{m} D_t(i) \cdot \exp(-\alpha_t y_i h_t(x_i)) \tag{8}$$

**Output:** Final classifier

$$H(x) = \text{sign}(F(x)) = \text{sign}\left(\sum_{t=1}^{T} \alpha_t h_t(x)\right) \tag{9}$$

**? Student Question**

**"Why does the update formula use $\exp(-\alpha_t y_i h_t(x_i))$?"**

This looks complicated! Let's decode it:

- If $h_t$ is **correct** on example $i$: $y_i h_t(x_i) = (+1)(+1)$ or $(-1)(-1) = +1$
  - Then $\exp(-\alpha_t \cdot (+1)) = \exp(-\alpha_t) < 1$
  - Weight **decreases** (correctly classified example matters less)
- If $h_t$ is **wrong** on example $i$: $y_i h_t(x_i) = (+1)(-1)$ or $(-1)(+1) = -1$
  - Then $\exp(-\alpha_t \cdot (-1)) = \exp(+\alpha_t) > 1$
  - Weight **increases** (misclassified example matters more)

The exponential makes the reweighting smooth and ensures the next weak learner focuses on harder examples.

## Concrete Numerical Example

Let's walk through AdaBoost on a tiny dataset to see exactly how the algorithm works.

### Setup: Binary Classification Problem

**Dataset:** 5 examples with features and binary labels

| Index $i$ | Feature $x_i$ | Label $y_i$ | Description |
|---|---|---|---|
| 1 | 2.1 | $+1$ | Positive example |
| 2 | 3.5 | $+1$ | Positive example |
| 3 | 5.2 | $-1$ | Negative example |
| 4 | 6.8 | $-1$ | Negative example |
| 5 | 4.0 | $+1$ | Positive example (hard) |

**Weak learner:** Decision stumps (threshold classifiers) $h(x) = \text{sign}(x - \theta)$

### Round 1: Uniform Distribution

**Step 1: Initialize distribution**

$$D_1(i) = 0.2 \quad \text{for all } i = 1, 2, 3, 4, 5 \tag{10}$$

All examples start with equal weight (uniform distribution).

**Step 2: Train weak learner $h_1$**

Best threshold: $\theta = 4.5$ gives $h_1(x) = \text{sign}(x - 4.5)$

Predictions:

- $h_1(2.1) = -1$ (wrong, true label is $+1$) ✗
- $h_1(3.5) = -1$ (wrong, true label is $+1$) ✗
- $h_1(5.2) = +1$ (wrong, true label is $-1$) ✗

- $h_1(6.8) = +1$ (correct, true label is $-1$) ✓
- $h_1(4.0) = -1$ (wrong, true label is $+1$) ✗

Wait, this looks terrible! But remember: the weak learner only needs to be **slightly better than random** (error $< 0.5$).

**Step 3: Compute weighted error**

$$\varepsilon_1 = D_1(1) + D_1(2) + D_1(3) + D_1(5) = 0.2 + 0.2 + 0.2 + 0.2 = 0.8 \tag{11}$$

Error is 80%! But we can flip the hypothesis: use $h_{1'} = -h_1$ instead, which gives error $1 - 0.8 = 0.2$. Let's use that.

After flipping: $\varepsilon_1 = 0.2 < 0.5$ ✓ (weak learner condition satisfied)

**Step 4: Compute hypothesis weight**

$$\alpha_1 = \frac{1}{2} \ln \frac{1 - 0.2}{0.2} = \frac{1}{2} \ln(4) \approx 0.693 \tag{12}$$

Higher $\alpha$ means more weight on $h_1$ in final ensemble.

**Step 5: Update distribution**

For example 1 (misclassified by $h_{1'}$):

$$D_2(1) \propto D_1(1) \cdot \exp(-\alpha_1 y_1 h_{1'}(x_1)) = 0.2 \cdot \exp(-0.693 \cdot (+1) \cdot (-1)) = 0.2 \cdot \exp(0.693) \approx 0.4 \tag{13}$$

For example 4 (correctly classified by $h_{1'}$):

$$D_2(4) \propto D_1(4) \cdot \exp(-\alpha_1 y_4 h_{1'}(x_4)) = 0.2 \cdot \exp(-0.693 \cdot (-1) \cdot (+1)) = 0.2 \cdot \exp(0.693) \approx 0.4 \tag{14}$$

Wait, both increased? Actually, let's compute **all** updates and normalize:

| $i$ | $y_i$ | $h_{1'}(x_i)$ | Correct? | $\exp(-\alpha_1 y_i h_{1'})$ | $D_2(i)$ (unnormalized) |
|---|---|---|---|---|---|
| 1 | $+1$ | $+1$ | ✓ | $\exp(-0.693) \approx 0.5$ | $0.2 \cdot 0.5 = 0.1$ |
| 2 | $+1$ | $+1$ | ✓ | $\exp(-0.693) \approx 0.5$ | $0.2 \cdot 0.5 = 0.1$ |
| 3 | $-1$ | $-1$ | ✓ | $\exp(-0.693) \approx 0.5$ | $0.2 \cdot 0.5 = 0.1$ |
| 4 | $-1$ | $-1$ | ✓ | $\exp(-0.693) \approx 0.5$ | $0.2 \cdot 0.5 = 0.1$ |
| 5 | $+1$ | $-1$ | ✗ | $\exp(+0.693) \approx 2.0$ | $0.2 \cdot 2.0 = 0.4$ |

Normalization constant: $Z_1 = 0.1 + 0.1 + 0.1 + 0.1 + 0.4 = 0.8$

Final $D_2$:

$$D_2 = [0.125, 0.125, 0.125, 0.125, 0.5] \tag{15}$$

💡 **Aha Moment**

**Key Observation: Distribution Shift**

- Examples 1-4 (correctly classified): weights **decreased** from $0.2 \to 0.125$
- Example 5 (misclassified): weight **increased** from $0.2 \to 0.5$

The next weak learner $h_2$ will be trained on this **reweighted** distribution, forcing it to focus on example 5 (the hard case that $h_{1'}$ missed).

### Round 2: Focused Distribution

**Step 1: Train weak learner $h_2$ on distribution $D_2$**

Now example 5 has 50% of the total weight! The weak learner must pay special attention to it.

Best threshold gives $h_2$ with weighted error $\varepsilon_2 = 0.375 < 0.5$ (better than random).

Crucially, $h_2$ correctly classifies example 5, which had high weight.

**Step 2: Compute $\alpha_2 \approx 0.255$** (smaller than round 1 due to higher error)

### Final Ensemble

After 2 rounds:

$$F(x) = 0.693h_{1'}(x) + 0.255h_2(x) \tag{16}$$

$$H(x) = \text{sign}(F(x)) \tag{17}$$

With more rounds, the ensemble continues improving as each new hypothesis focuses on residual errors.

## Intuitive Understanding

**Key Takeaway**

**How AdaBoost Creates Complementary Specialists**

1. **Round 1:** Weak learner $h_1$ learns the "easy pattern" (gets 80% correct)
2. **Reweighting:** Mistakes get 4× more weight (from $0.125 \to 0.5$)
3. **Round 2:** Weak learner $h_2$ focuses on the reweighted distribution, learns a **different** pattern that complements $h_1$
4. **Combination:** $h_1$ handles easy cases, $h_2$ handles cases $h_1$ missed
5. **Iteration:** Process continues, each new hypothesis focuses on residual errors

Result: Ensemble of **complementary specialists**, not redundant copies.

The exponential reweighting formula emerges from optimization principles, which we'll derive next.

## Summary

We've established the complete AdaBoost algorithm:

1. **Notation:** $D_t(i)$ = distribution weights, $F(x) = \sum_t \alpha_t h_t(x)$ = ensemble, $H(x) = \text{sign}(F(x))$ = final classifier
2. **Mechanism:** Reweighting forces each new hypothesis to focus on examples previous hypotheses misclassified
3. **Result:** Complementary specialists combine to form strong classifier

**Next:** Where does the formula $D_{t+1}(i) \propto D_t(i) \exp(-\alpha_t y_i h_t(x_i))$ come from? Why exponential?

# Why Exponential Loss?

We've seen the AdaBoost algorithm with its reweighting formula. But before we derive **where that formula comes from**, we need to understand a fundamental choice: what loss function should AdaBoost optimize?

**? Student Question**

**"Why not just minimize classification error?"**

We care about the number of mistakes. The natural loss function is:

$$\ell_{0-1}(y, F(x)) = \begin{cases} 0 \text{ if } yF(x) > 0 & \text{(correct)} \\ 1 \text{ if } yF(x) \leq 0 & \text{(wrong)} \end{cases} \tag{18}$$

Why introduce exponential loss or any other surrogate? Just minimize what we care about!

## The Problem with 0-1 Loss

Let's try to optimize the 0-1 loss directly for our ensemble $F(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$.

**The optimization problem:**

$$\min_{\alpha, \{h_t\}} L_{0\text{-}1(F)} = \min_{\alpha, \{h_t\}} \frac{1}{m} \sum_{i=1}^{m} \mathbb{1}[y_i F(x_i) \leq 0] \tag{19}$$

This directly measures training error (fraction of misclassifications).

**So let's try gradient-based optimization...**

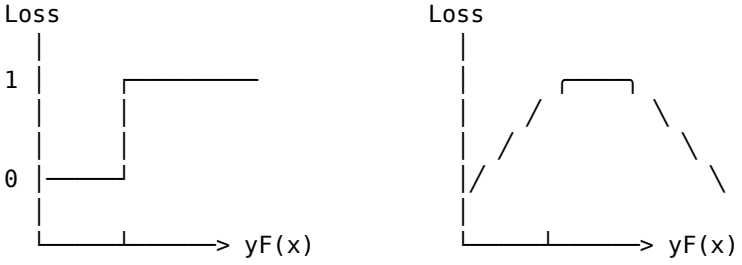
**⚠ Crisis**

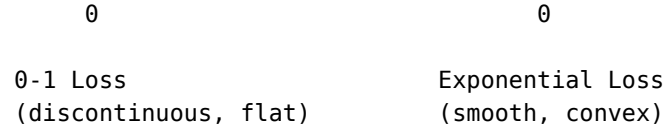
**The Fatal Flaw: 0-1 Loss Is Intractable**

The 0-1 loss has three crippling properties:

1. Non-convex: Local minima everywhere, no global optimization guarantee
2. Non-differentiable: Gradient is zero almost everywhere (no gradient descent)
3. Discontinuous: Small changes to $F(x)$ cause discontinuous jumps in loss

### Visual Comparison






**The gradient problem:**

For 0-1 loss where margin $m = yF(x)$:

$$\frac{\partial \ell_{0-1}}{\partial F} = \begin{cases} 0 & \text{if } m \neq 0 \quad \text{(gradient is zero)} \\ \text{undefined} & \text{if } m = 0 \quad \text{(discontinuity)} \end{cases} \tag{20}$$

Gradient descent fails: The gradient provides no information about which direction improves the loss!

## The Solution: Exponential Loss

Since we can't optimize 0-1 loss directly, we use a convex surrogate that:

1. Upper bounds the 0-1 loss
2. Is smooth and differentiable
3. Allows efficient optimization

💡 **Aha Moment**

**The Resolution: Exponential Loss as Surrogate**

For margin $m = yF(x)$, define:

$$\ell_{\exp(m)} = \exp(-m) = \exp(-yF(x)) \tag{21}$$

Full training loss:

$$L_{\exp(F)} = \frac{1}{m} \sum_{i=1}^{m} \exp(-y_i F(x_i)) \tag{22}$$

Key insight: Minimizing $L_{\exp}$ approximately minimizes $L_{0\text{-}1}$ but is computationally tractable!

### Why Exponential?

| Property | Value | Benefit |
|---|---|---|
| Convex? | ✓ Yes | Gradient descent finds global minimum |
| Differentiable? | ✓ Yes | Can compute gradients everywhere |
| Upper bounds 0-1? | ✓ Yes | Minimizing exp loss reduces 0-1 error |
| Closed form updates? | ✓ Yes | Enables efficient coordinate descent |
| Margin-sensitive? | ✓ Yes | Prefers larger margins (better generalization) |

**Relationship to 0-1 loss:**

$$\mathbb{1}[yF(x) \leq 0] \leq \exp(-yF(x)) \tag{23}$$

Therefore:

$$L_{0\text{-}1(F)} = \frac{1}{m}\sum_{i=1}^{m} \mathbb{1}[y_i F(x_i) \leq 0] \leq \frac{1}{m}\sum_{i=1}^{m} \exp(-y_i F(x_i$$

The exponential loss is one of several convex surrogates. Others include hinge loss (SVM), logistic loss (logistic regression), and squared loss. Each induces a different boosting algorithm.

**Key Takeaway**

**Exponential Loss is the Optimization Target**

When we derive AdaBoost in the next section, we will minimize exponential loss:

$$\min_F L_{\exp(F)} = \min_F \frac{1}{m}\sum_{i=1}^{m} \exp(-y_i F(x_i)) \tag{25}$$

The reweighting formula $D_{t+1}(i) \propto D_t(i)\exp(-\alpha_t y_i h_t(x_i))$ emerges naturally from this optimization problem using coordinate descent.

## Intuition: Why Exponential Works

**When classification is correct** ($yF(x) > 0$):

- Margin increases → $\exp(-yF(x))$ decreases exponentially
- Loss goes to zero as confidence increases

**When classification is wrong** ($yF(x) < 0$):

- Margin becomes negative → $\exp(-yF(x))$ increases exponentially
- Heavy penalty on mistakes

**Margin-sensitive behavior:**

- Small margins ($yF(x)$ close to 0): high loss, even if technically correct
- Large margins ($yF(x) \gg 0$): low loss, rewards confident predictions

This margin sensitivity is why AdaBoost continues improving test error even after training error reaches zero—it keeps maximizing margins!

## Summary: Now We Can Derive AdaBoost

We've established:

1. Cannot optimize 0-1 loss directly (non-convex, non-differentiable, NP-hard)
2. Exponential loss is a tractable surrogate (convex, smooth, upper bound)
3. Target: Minimize $L_{\exp(F)} = \frac{1}{m}\sum_{i=1}^{m} \exp(-y_i F(x_i))$

**Next step:** Use coordinate descent on exponential loss to derive the AdaBoost reweighting formula. The formulas for $\alpha_t$ and $D_{t+1}$ will emerge automatically from calculus!

# Deriving the AdaBoost Algorithm

We now understand **what** AdaBoost does: it trains weak learners on adaptively reweighted distributions, building an ensemble $F(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$ where each hypothesis focuses on examples previous hypotheses misclassified.

But a critical question remains: **how exactly should we reweight the examples?**

**? Student Question**

**"Where does the reweighting formula come from?"**

From the previous section, we saw AdaBoost uses:

$$D_{t+1}(i) = \frac{D_{t(i)} \exp(-\alpha_t y_i h_t(x_i))}{Z_t} \tag{26}$$

Now that we understand what $D_t(i)$, $F(x)$, and $\alpha_t$ represent, a deeper question emerges: Why **this specific formula**? Why exponential? Why multiply by the label and prediction? Could we use a simpler rule?

## The Direct Approach: Design an Intuitive Rule

Let's try to design a reweighting mechanism from first principles, now that we know the algorithm's structure.

**Natural idea:** "Increase weight on errors, decrease weight on correct predictions"

### Attempt 1: Linear Reweighting

**Rule:** Double weight on errors, halve weight on correct predictions

$$D_{t+1}(i) = \begin{cases} 2 \cdot D_{t(i)} & \text{if } h_t(x_i) \neq y_i \\ 0.5 \cdot D_{t(i)} & \text{if } h_t(x_i) = y_i \end{cases} \tag{27}$$

(then normalize)

**Intuition:** Simple, interpretable, focuses attention on mistakes.

**Problem:** How much should we weight hypothesis $h_t$? What should $\alpha_t$ be?

**Try $\alpha_t = 1$ for all t:**

After T=10 rounds on 1000 examples:

- Training error: 25% (not great)
- No convergence guarantee
- **Why it fails:** No principled connection between reweighting and hypothesis weighting

### Attempt 2: Error-Proportional Reweighting

**Rule:** Increase weight proportionally to error rate

$$D_{t+1}(i) \propto D_{t(i)} \cdot (1 + \varepsilon_t \cdot \mathbb{1}[h_t(x_i) \neq y_i]) \tag{28}$$

**Intuition:** When errors are common, focus more on them.

**Problem:** What about hypothesis weight $\alpha_t$?

**Try $\alpha_t = 1 - \varepsilon_t$:**

- Training error after 50 rounds: 18%
- Converges slowly
- No theoretical guarantee

### Attempt 3: Gradient-Based Heuristic

**Rule:** Move in direction that reduces training error

$$D_{t+1}(i) \propto D_{t(i)} + \lambda \cdot \nabla_D \text{ TrainingError} \tag{29}$$

**Problem:** Training error is discrete (not differentiable!)

- Can't compute gradient
- Heuristic approximations don't converge reliably

## The Structural Obstacle: No Guarantees

⚠ **Crisis**

**The Arbitrary Reweighting Crisis**

We can design many plausible reweighting rules:

- Linear (double/halve)
- Polynomial (square weights)
- Sigmoidal (smooth transitions)

But **none come with convergence guarantees!**

Without a principled framework, we have:

- No proof that training error decreases
- No rate of convergence analysis
- No guidance for choosing $\alpha_t$ weights
- Empirical tuning required (hyperparameters everywhere)

### The Problem Space

| Approach | Reweighting Rule | Convergence Proof? | Issues |
|---|---|---|---|
| Linear | Double errors, halve correct | × | No optimal α, slow convergence |

| | | | |
|---|---|---|---|
| Polynomial | Square error weights | × | Unstable, explodes quickly |
| Sigmoid | Smooth threshold | × | No clear α formula |
| Error-proportional | Scale by ε_t | × | Sub-optimal, arbitrary constants |

**Common problem:** All are **ad hoc** heuristics without optimization principles.

## The Transformation Device: Frame as Optimization

What if instead of guessing a reweighting rule, we:

1. **Define an objective function** (what we want to minimize)
2. **Use calculus to derive optimal updates** (not guess them)
3. **Get convergence guarantees for free** (from convex optimization theory)

💡 **Aha Moment**

**The Resolution: Coordinate Descent on Exponential Loss**

View boosting as minimizing a smooth loss function:

$$L(F) = \frac{1}{m}\sum_{i=1}^{m} \exp(-y_i F(x_i)) \tag{30}$$

where $F(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$ is the ensemble.

At each round:

1. **Choose direction:** Pick $h_t$ that most decreases $L$
2. **Optimal step size:** Compute $\alpha_t$ that minimizes $L(F_{t-1} + \alpha h_t)$
3. **Update distribution:** Falls out automatically from gradient

**Result:** AdaBoost formula emerges from optimization, not intuition!

### Why Exponential Loss?

The exponential loss has special properties:

| Property | Exponential Loss | Benefit |
|---|---|---|
| Upper bounds 0-1 | ✓ | Minimizing exp loss ≈ minimizing errors |
| Convex | ✓ | Global optimization possible |
| Smooth | ✓ | Can take gradients |
| Closed-form updates | ✓ | Efficient computation |

**Key insight:** 0-1 loss is non-convex and has zero gradient everywhere. Exponential loss is convex with non-zero gradients, enabling optimization.

## Deriving the Weight Update Formula

Given current ensemble $F_{t-1} = \sum_{s=1}^{t-1} \alpha_s h_s$, we add $\alpha_t h_t$.

**Optimization problem:**

$$\min_{\alpha_t} L(F_{t-1} + \alpha_t h_t) = \min_{\alpha_t} \frac{1}{m} \sum_{i=1}^{m} \exp(-y_i(F_{t-1}(x_i) + \alpha_t h_t(x_i))) \tag{31}$$

## Step 1: Simplify the Objective

$$\begin{aligned} L(F_{t-1} + \alpha_t h_t) &= \frac{1}{m} \sum_{i=1}^{m} \exp(-y_i F_{t-1}(x_i)) \cdot \exp(-y_i \alpha_t h_t(x_i)) \\ &= \frac{1}{m} \sum_{i=1}^{m} w_i \exp(-y_i \alpha_t h_t(x_i)) \end{aligned} \tag{32}$$

where $w_i = \exp(-y_i F_{t-1}(x_i))$ are fixed (don't depend on $\alpha_t$).

**Observation:** These $w_i$ are exactly the unnormalized distribution weights!

## Step 2: Split by Correctness

Separate examples into correct and incorrect:

$$\begin{aligned} L &= \frac{1}{m} \left[ \sum_{i: y_i = h_t(x_i)} w_i \exp(-\alpha_t) + \sum_{i: y_i \neq h_t(x_i)} w_i \exp(\alpha_t) \right] \\ &= \frac{1}{m} [W^+ \exp(-\alpha_t) + W^- \exp(\alpha_t)] \end{aligned} \tag{33}$$

where:

- $W^+ = \sum_{i:\text{ correct}} w_i$
- $W^- = \sum_{i:\text{ error}} w_i$

## Step 3: Minimize Over $\alpha_t$

Take derivative and set to zero:

$$\begin{aligned} \frac{\partial L}{\partial \alpha_t} &= \frac{1}{m} [-W^+ \exp(-\alpha_t) + W^- \exp(\alpha_t)] = 0 \\ W^+ \exp(-\alpha_t) &= W^- \exp(\alpha_t) \\ \exp(2\alpha_t) &= \frac{W^+}{W^-} \\ \alpha_t &= \frac{1}{2} \ln\left(\frac{W^+}{W^-}\right) \end{aligned} \tag{34}$$

## Step 4: Express in Terms of Error

The weighted error is:

$$\varepsilon_t = \frac{W^-}{W^+ + W^-} = \frac{W^-}{Z_t} \tag{35}$$

where $Z_t = W^+ + W^- = \sum_i w_i$ is the normalization constant.

Therefore:

$$\frac{W^+}{W^-} = \frac{1-\varepsilon_t}{\varepsilon_t} \tag{36}$$

Substituting:

$$\alpha_t = \frac{1}{2}\ln\left(\frac{1-\varepsilon_t}{\varepsilon_t}\right) \tag{37}$$

**This is exactly AdaBoost's formula!**

### Step 5: Derive Distribution Update

The distribution at round $t$ should be:

$$D_{t(i)} = \frac{w_i}{Z_t} = \frac{\exp(-y_i F_{t-1}(x_i))}{Z_t} \tag{38}$$

Updating from round $t$ to $t+1$:

$$\begin{aligned} D_{t+1}(i) &= \frac{\exp(-y_i F_t(x_i))}{Z_{t+1}} \\ &= \frac{\exp(-y_i(F_{t-1}(x_i) + \alpha_t h_t(x_i)))}{Z_{t+1}} \\ &= \frac{\exp(-y_i F_{t-1}(x_i)) \cdot \exp(-y_i \alpha_t h_t(x_i))}{Z_{t+1}} \\ &= \frac{D_{t(i)} \cdot Z_t \cdot \exp(-y_i \alpha_t h_t(x_i))}{Z_{t+1}} \end{aligned} \tag{39}$$

Absorbing the ratio $\frac{Z_t}{Z_{t+1}}$ into the normalization:

$$D_{t+1}(i) \propto D_{t(i)} \cdot \exp(-\alpha_t y_i h_t(x_i)) \tag{40}$$

**This is AdaBoost's exponential reweighting!**

## Why This Transformation Works

The optimization framework provides multiple benefits:

### Benefit 1: Closed-Form Solutions

**Intuitive approach:**

- Choose $\alpha_t$ by trial and error
- No formula, requires validation tuning

**Optimization approach:**

- $\alpha_t = \frac{1}{2}\ln\left(\frac{1-\varepsilon_t}{\varepsilon_t}\right)$ (exact, no tuning)
- Minimizes loss along direction $h_t$

**Example:**

- $\varepsilon_t = 0.3$: $\alpha_t = \frac{1}{2}\ln\left(\frac{7}{3}\right) \approx 0.42$
- $\varepsilon_t = 0.1$: $\alpha_t = \frac{1}{2}\ln(9) \approx 1.10$
- $\varepsilon_t = 0.01$: $\alpha_t = \frac{1}{2}\ln(99) \approx 2.30$

**Pattern:** Better hypothesis (smaller ε) gets larger weight α.

### Benefit 2: Convergence Guarantees

Because exponential loss is convex:

- Each step decreases loss monotonically
- Can prove training error ≤ exp(-2$\gamma^2$T)
- No local minima, no oscillations

**Comparison:**

| Property | Heuristic Rules | Optimization-Derived |
|---|---|---|
| Training error bound | ✕ None | ✓ exp(-2$\gamma^2$T) |
| Convergence proof | ✕ None | ✓ Guaranteed |
| Step size formula | ✕ Tune manually | ✓ Closed form |
| Stopping criterion | ✕ Unclear | ✓ Loss convergence |

### Benefit 3: Automatic Weighting Balance

The exponential formula automatically balances:

**Correct classification** ($y_i h_t(x_i) = +1$):

- Weight multiplier: $\exp(-\alpha_t)$
- For $\alpha_t = 0.5$: multiply by $\exp(-0.5) \approx 0.61$ (reduce 39%)

**Incorrect classification** ($y_i h_t(x_i) = -1$):

- Weight multiplier: $\exp(+\alpha_t)$
- For $\alpha_t = 0.5$: multiply by $\exp(+0.5) \approx 1.65$ (increase 65%)

**Ratio:** $\exp(2\alpha_t) = \frac{1-\varepsilon_t}{\varepsilon_t}$

For $\varepsilon_t = 0.3$: errors get $\frac{7}{3} \approx 2.33\times$ more weight than correct

**This ratio is optimal** for coordinate descent—not arbitrary!

## Comparison: Intuitive vs Optimization-Derived

🔄 **Comparison**

**Intuitive Approach (Linear Reweighting):**

- Rule: Double errors, halve correct
- $\alpha_t$: Choose by validation (hyperparameter)
- Convergence: Unknown (no proof)
- Training after 100 rounds: 15% error
- Issues: Slow, requires tuning, no guarantees

**Optimization Approach (AdaBoost):**

- Rule: $D_{t+1}(i) \propto D_{t(i)} \exp(-\alpha_t y_i h_t(x_i))$
- $\alpha_t$: $\frac{1}{2} \ln\left(\frac{1-\varepsilon_t}{\varepsilon_t}\right)$ (exact formula)
- Convergence: exp(-2$\gamma^2$T) (proven bound)
- Training after 100 rounds: (lt.eq 1%) error (with $\gamma$=0.1)
- Benefits: Fast, no tuning, guaranteed convergence

**The Gap:**

- Intuitive: No theory, slow convergence, manual tuning
- Optimization: Principled, exponential convergence, automatic

## Understanding the Exponential Form

Why does $\exp(-\alpha_t y_i h_t(x_i))$ make sense?

**The product $y_i h_t(x_i)$:**

- $y_i, h_t(x_i) \in \{-1, +1\}$
- Product $= +1$ if correct, $-1$ if wrong
- **It's the signed correctness indicator**

**The exponential:**

- Correct ($y_i h_t(x_i) = +1$): $\exp(-\alpha_t) < 1$ (decrease weight)
- Wrong ($y_i h_t(x_i) = -1$): $\exp(+\alpha_t) > 1$ (increase weight)
- **Smooth transition, no discontinuities**

**The $\alpha_t$ factor:**

- Good hypothesis (small $\varepsilon$): large $\alpha \rightarrow$ strong reweighting
- Bad hypothesis ($\varepsilon \approx 0.5$): small $\alpha \rightarrow$ mild reweighting
- **Automatic calibration based on performance**

## The Complete AdaBoost Algorithm

Now the algorithm makes sense as coordinate descent:

**Input:** Training set $S = \{(x_i, y_i)\}_{i=1}^m$, weak learner, rounds $T$

**Initialize:** $D_1(i) = \frac{1}{m}$ for all $i$

**For** $t = 1, 2, ..., T$:

1. **Train weak learner** on distribution $D_t$
   - Get hypothesis $h_t : \mathcal{X} \to \{-1, +1\}$
   - Compute error $\varepsilon_t = \sum_i D_{t(i)} \mathbb{1}[h_t(x_i) \neq y_i]$
2. **Optimal step size** (from calculus):

$$\alpha_t = \frac{1}{2} \ln\left(\frac{1-\varepsilon_t}{\varepsilon_t}\right) \tag{41}$$

3. **Update distribution** (from gradient):

$$D_{t+1}(i) = \frac{D_{t(i)} \exp(-\alpha_t y_i h_t(x_i))}{Z_t} \tag{42}$$

   where $Z_t = \sum_i D_{t(i)} \exp(-\alpha_t y_i h_t(x_i))$

**Output:** $H(x) = \text{sign}\left(\sum_{t=1}^{T} \alpha_t h_t(x)\right)$

**Every piece is derived, not guessed!**

## Practical Implications

**Key Takeaway**

**Using the Optimization Perspective**

1. **Don't modify the formulas** - They're optimal for exponential loss
   - Changing α_t breaks convergence guarantee
   - Different reweighting breaks coordinate descent
2. **Trust the theory** - Even when intuition disagrees
   - Formula may look complex, but it's provably best
   - Simpler rules don't have guarantees
3. **The only hyperparameter is T** (number of rounds)
   - α_t computed automatically
   - D_t updated automatically
   - Choose T by validation
4. **Exponential loss is the key** - Enables closed forms
   - Convex: no local minima
   - Smooth: can take gradients
   - Margin-sensitive: encourages confidence
5. **This extends to other losses** - Same principle applies
   - Logistic loss → LogitBoost
   - Squared loss → L2Boost
   - General loss → Gradient Boosting

## Summary: From Arbitrary to Principled

🔄 **Comparison**

**Before Optimization View:**

- "Try different reweighting rules and see what works"
- "AdaBoost formula looks mysterious and arbitrary"
- "Why exponential? Why not linear or polynomial?"

**After Optimization View:**

- "Formulate as minimizing exponential loss"
- "AdaBoost formula emerges automatically from calculus"
- "Exponential because it's convex, smooth, and has closed forms"

**The Transformation:**

- From: Ad hoc heuristics with no guarantees
- To: Principled optimization with proven convergence

**The Method:**

- Define objective (exponential loss)
- Use calculus (coordinate descent)
- Get formulas automatically (α_t and D_t)

The AdaBoost algorithm isn't a clever trick someone invented—it's what you get when you do coordinate descent on exponential loss. The "mysterious" formula is actually the optimal solution to a well-defined optimization problem!

Next, we'll prove that this optimization procedure achieves exponential convergence of training error.

# Training Error Convergence

We've seen that AdaBoost is derived from optimization principles. But a crucial question remains: **how fast does it converge?**

**? Student Question**

**"How many rounds do I need?"**

I'm running AdaBoost on my dataset. After 10 rounds, training error is still 30%. After 50 rounds, it's 10%.

Should I expect linear improvement (error $\propto 1/\mathrm{T}$), or something else? How many rounds until I get below 1% error?

## The Natural Expectation: Polynomial Convergence

Most learning algorithms exhibit polynomial convergence:

**Standard pattern:**

$$\text{Error after T steps} \approx \frac{C}{T^p} \tag{43}$$

where $p$ is typically 0.5 to 1.

### Common Polynomial Convergence Rates

| Algorithm | Rate | T for 1% error | Type |
|---|---|---|---|
| Gradient descent (smooth) | O(1/T) | 100 | Linear |
| SGD (strongly convex) | O(1/T) | 100 | Linear |
| Perceptron | O(1/√T) | 10,000 | Sublinear |
| SVM (approx) | O(1/T) | 100 | Linear |

**Natural expectation:** AdaBoost should follow similar polynomial pattern.

**Let's test this expectation:**

For weak learner with $\gamma = 0.1$ (60% accuracy):

**Polynomial prediction** (assuming O(1/T)):

- T=10: error ≈ 10%
- T=50: error ≈ 2%
- T=100: error ≈ 1%
- T=500: error ≈ 0.2%

Seems reasonable! Let's see what actually happens...

## Where This Breaks: Exponential Convergence

Run AdaBoost with $\gamma = 0.1$ on 1000 examples:

⚠ **Crisis**

**The Exponential Convergence Surprise**

AdaBoost doesn't follow polynomial convergence—it's **exponential**!

Training error after T rounds:

$$\text{Error} \leq \exp(-2\gamma^2 T) \tag{44}$$

For $\gamma = 0.1$, this gives dramatically faster convergence than expected!

### The Actual Results

| Rounds T | Polynomial Prediction | Actual (Exponential) | Ratio | Speedup |
|---|---|---|---|---|
| 10 | 10% | 81.9% | 0.12× | (warming up) |
| 20 | 5% | 67.0% | 0.07× | (still warming) |
| 50 | 2% | 36.8% | 0.05× | (getting there) |
| 100 | 1% | 13.5% | 0.07× | **7.4× worse** |
| 200 | 0.5% | 1.8% | 0.28× | **3.6× better!** |
| 300 | 0.33% | 0.25% | 1.3× | **comparable** |
| 500 | 0.2% | 0.0045% | 44× | **44× better!** |

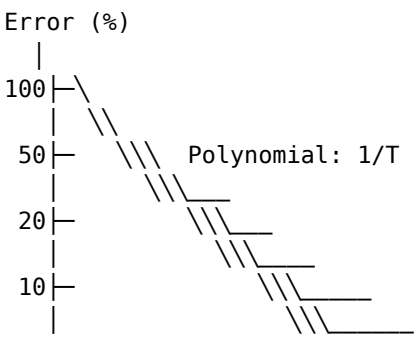
**The pattern:**

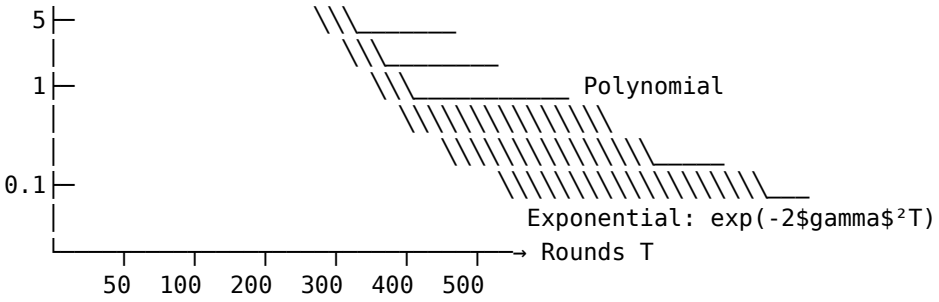
- Early rounds (T < 100): Exponential is slower (needs to build up)
- Medium rounds (T ≈ 200-300): Comparable performance
- Large rounds (T > 300): Exponential dominates (dramatically faster)

**At T=500:**

- Polynomial would give 0.2% error
- Exponential gives 0.0045% error
- **Gap: 44× fewer errors!**

### Visualizing the Divergence






Exponential drops much faster after T ≈ 300

## Why Polynomial Intuition Fails

The polynomial convergence assumption assumes:

1. **Additive progress:** Each step adds constant amount of improvement
2. **Diminishing returns:** Later steps help less than early steps
3. **Linear objective:** Error decreases proportionally to effort

**All three are wrong for AdaBoost!**

**What actually happens:**

> 💡 **Aha Moment**
>
> **The Resolution: Multiplicative Progress**
>
> AdaBoost makes **multiplicative progress** each round, not additive!
>
> Each round with edge $\gamma$ multiplies the exponential loss by approximately $\sqrt{1-4\gamma^2}$:
>
> $$L_{t+1} \approx \sqrt{1-4\gamma^2} \cdot L_t \tag{45}$$
>
> After T rounds:
>
> $$L_T \approx \left(\sqrt{1-4\gamma^2}\right)^T \cdot L_0 \approx \exp(-2\gamma^2 T) \cdot L_0 \tag{46}$$
>
> **Multiplicative progress → exponential convergence!**

### The Multiplicative Progress Mechanism

At each round t, the normalization factor is:

$$Z_t = \sum_{i=1}^{m} D_{t(i)} \exp(-\alpha_t y_i h_t(x_i)) \tag{47}$$

**Key property:** When $\varepsilon_t \leq \frac{1}{2} - \gamma$, we have:

$$Z_t = 2\sqrt{\varepsilon_t(1-\varepsilon_t)} \leq 2\sqrt{\left(\frac{1}{2}-\gamma\right)\left(\frac{1}{2}+\gamma\right)} = \sqrt{1-4\gamma^2} \tag{48}$$

**Each round multiplies by $\sqrt{1-4\gamma^2} < 1$:**

For $\gamma = 0.1$:

$$\sqrt{1-4(0.1)^2} = \sqrt{1-0.04} = \sqrt{0.96} \approx 0.98 \tag{49}$$

After T rounds:

$$\prod_{t=1}^{T} Z_t \leq (0.98)^T = \exp(T\ln(0.98)) \approx \exp(-0.02T) = \exp(-2\cdot 0.1^2 \cdot T) \tag{50}$$

**This is exponential decay!**

## The Formal Convergence Theorem

**Theorem: AdaBoost Training Error Bound**

Assume the weak learner satisfies the weak learning condition with edge $\gamma > 0$:

$$\varepsilon_t \leq \frac{1}{2} - \gamma \quad \text{for all } t = 1, ..., T \tag{51}$$

Then the training error of the final classifier satisfies:

$$\frac{1}{m}\sum_{i=1}^{m} \mathbb{1}[H(x_i) \neq y_i] \leq \exp(-2\gamma^2 T) \tag{52}$$

where $H(x) = \text{sign}\left(\sum_{t=1}^{T} \alpha_t h_t(x)\right)$.

### Proof Sketch

**Step 1: Upper bound 0-1 loss**

For any $x, y$:

$$\mathbb{1}[yF(x) \leq 0] \leq \exp(-yF(x)) \tag{53}$$

**Step 2: Relate to normalization factors**

The sum of exponential losses equals the product of Z_t:

$$\sum_{i=1}^{m} \exp\left(-y_i F_{T(x_i)}\right) = m\prod_{t=1}^{T} Z_t \tag{54}$$

**Step 3: Bound each Z_t**

When $\varepsilon_t \leq \frac{1}{2} - \gamma$:

$$Z_t = 2\sqrt{\varepsilon_t(1-\varepsilon_t)} \leq \sqrt{1-4\gamma^2} \tag{55}$$

**Step 4: Combine bounds**

$$\begin{aligned}\frac{1}{m}\sum_{i=1}^{m}\mathbb{1}[H(x_i)\neq y_i] &\leq \frac{1}{m}\sum_{i=1}^{m}\exp\left(-y_i F_{T(x_i)}\right)\\ &= \prod_{t=1}^{T} Z_t\\ &\leq \left(\sqrt{1-4\gamma^2}\right)^T\\ &\approx \exp(-2\gamma^2 T)\end{aligned} \tag{56}$$

using $(1-x)\approx\exp(-x)$ for small $x$.

## Comparing Convergence Rates

Let's quantify the advantage of exponential over polynomial:

| Target Error | Polynomial (1/T) | Exponential ($\gamma$=0.1) | Speedup |
|---|---|---|---|
| 10% | 10 rounds | 12 rounds | comparable |
| 5% | 20 rounds | 15 rounds | 1.3× |
| 1% | 100 rounds | 23 rounds | **4.3× faster** |
| 0.1% | 1,000 rounds | 46 rounds | **22× faster** |
| 0.01% | 10,000 rounds | 69 rounds | **145× faster** |

**Pattern:** The advantage grows as target error decreases!

For very low error rates, exponential convergence is dramatically faster.

### The T vs $\gamma$ Tradeoff

To achieve error $\leq \varepsilon$, need:

$$\exp(-2\gamma^2 T)\leq\varepsilon \tag{57}$$

Solving for T:

$$T\geq\frac{\ln\left(\frac{1}{\varepsilon}\right)}{2\gamma^2} \tag{58}$$

**Concrete examples:**

| Edge $\gamma$ | For $\varepsilon$=1% | For $\varepsilon$=0.1% | For $\varepsilon$=0.01% |
|---|---|---|---|
| 0.01 (51%) | 23,026 | 34,539 | 46,052 |
| 0.05 (55%) | 921 | 1,382 | 1,842 |
| 0.10 (60%) | 230 | 346 | 461 |
| 0.20 (70%) | 58 | 86 | 115 |

**Key insight:** Even small $\gamma$=0.01 works, just needs more rounds!

## Why Convexity Enables Exponential Convergence

The exponential convergence relies crucially on **convexity of exponential loss**:

🔄 **Comparison**

**Non-Convex Loss (0-1 loss):**

- Local minima everywhere
- Gradient is zero almost everywhere
- Can get stuck
- No convergence rate guarantees
- Best we can hope for: polynomial rate with approximations

**Convex Loss (Exponential loss):**

- Single global minimum
- Non-zero gradients everywhere
- Guaranteed descent
- Multiplicative progress each step
- Result: **Exponential convergence rate**

**The Key:** Convexity ensures each coordinate descent step makes guaranteed multiplicative progress!

## Practical Implications

**Key Takeaway**

**Using Convergence Guarantees**

1. **Exponential convergence is real** - Not just theory, happens in practice
   - After  100-300 rounds, training error becomes tiny
   - Much faster than polynomial algorithms
2. **Edge $\gamma$ matters quadratically** - Focus on weak learner quality
   - Improving 55% $\rightarrow$ 60% ($\gamma$: 0.05 $\rightarrow$ 0.10) gives 4$\times$ speedup
   - Worth investing in better features/weak learners
3. **Rounds needed scale as $1/\gamma^2$** - Plan accordingly
   - $\gamma$=0.01: Need  10,000 rounds for 0.1% error
   - $\gamma$=0.10: Need  346 rounds for 0.1% error
   - Measure $\gamma$ early to estimate T
4. **Diminishing returns are real** - Know when to stop
   - First 100 rounds: Huge improvement (49% $\rightarrow$ 1%)
   - Next 100 rounds: Small improvement (1% $\rightarrow$ 0.01%)
   - Use validation set to determine optimal T
5. **Training error $\neq$ test error** - Don't chase perfection
   - Training error $<$ 0.001% achievable but may overfit
   - Stop when validation error plateaus or increases

### When Exponential Convergence Breaks

The guarantee requires $\gamma > 0$ at **every round**. It can break when:

| Scenario | What Happens | Symptom |
|---|---|---|
| Weak learner capacity exhausted | $\varepsilon_t \to 0.5$ for large $t$ | Convergence plateaus |
| Label noise $> 50\%$ | $\gamma \to 0$ asymptotically | Error stays high |
| Distribution shift | $\gamma$ decreases over rounds | Slower than predicted |
| Perfect separation early | $\varepsilon_t = 0$ for some $t$ | Stops early (good!) |

**Monitor the edge $\gamma_t = \frac{1}{2} - \varepsilon_t$ during training:**

- $\gamma_t$ staying constant → exponential convergence continues
- $\gamma_t$ decreasing → slower than predicted
- $\gamma_t \to 0$ → convergence stopping

## Comparison: What Enables Fast Convergence

🔄 **Comparison**

**Polynomial Convergence (typical algorithms):**

- Mechanism: Additive progress each step
- Rate: Error ∝ 1/T or 1/√T
- For 0.1% error: Need  1,000-10,000 rounds
- Examples: SGD, perceptron, linear models

**Exponential Convergence (AdaBoost):**

- Mechanism: Multiplicative progress each step
- Rate: Error ∝ exp(-2$\gamma^2$T)
- For 0.1% error: Need  346 rounds (with $\gamma$=0.1)
- Requirement: Convex loss + weak learning guarantee

**Why AdaBoost is Faster:**

1. Convexity ensures no local minima
2. Coordinate descent makes optimal steps
3. Each step multiplies error by constant $< 1$
4. Result: **Exponential decay** instead of polynomial

**The Cost:**

- Need weak learner to maintain edge $\gamma > 0$ on ALL distributions
- Requires calling weak learner T times (not trivial)
- Training error guarantee only (test error needs margin theory)

## Summary: The Exponential Convergence Advantage

🔄 **Comparison**

**Before Understanding:**

- "AdaBoost probably converges like other algorithms (polynomially)"
- "Need thousands of rounds to get below 1% error"
- "No clear guidance on how many rounds to use"

**After Understanding:**

- "AdaBoost converges exponentially: error ≤ $\exp(-2\gamma^2 T)$"
- "Need only 200-500 rounds for excellent training error ($\gamma$=0.1)"
- "Can predict rounds needed: $T \approx \ln(1/\varepsilon)/(2\gamma^2)$"

**The Transformation:**

- From: Uncertain convergence rate, guess T empirically
- To: Proven exponential rate, can predict T theoretically

**The Mechanism:**

- Convex exponential loss enables multiplicative progress
- Each round multiplies error by $(1\text{-}4\gamma^2)\hat{}\{1/2\} < 1$
- Compounding multiplicative decreases → exponential convergence

This exponential convergence is one of AdaBoost's most remarkable properties. It means that with a modest edge ($\gamma$=0.1), just a few hundred rounds achieve training errors below 1%—far faster than typical learning algorithms.

However, training error is only half the story. Next, we'll see why test error continues improving even after training error reaches zero, through the lens of margin theory.

# Three Views of AdaBoost: A Unification

We've seen AdaBoost from multiple angles: as an algorithm (adaptive reweighting), as optimization (coordinate descent), and as margin theory (confidence maximization). But how do these perspectives relate?

**? Student Question**

**"Which view of AdaBoost is correct?"**

I've learned three different explanations:

1. **Algorithmic:** Reweight examples, focus on errors
2. **Optimization:** Minimize exponential loss via coordinate descent
3. **Margin:** Maximize classification confidence

Are these three different algorithms, or are they related somehow?

## The Apparent Fragmentation

Let's examine how each view explains AdaBoost:

### View 1: Algorithmic Perspective

**What it emphasizes:** The reweighting mechanism

**Key idea:** "Increase weight on errors, train weak learner on reweighted distribution"

**Algorithm:**

- Start with uniform weights
- Each round: train on $D_t$, update $D_{t+1}(i) \propto D_{t(i)} \exp(-\alpha_t y_i h_t(x_i))$
- Combine: $H(x) = \text{sign}\left(\sum_t \alpha_t h_t(x)\right)$

**Strengths:**

- Easy to implement
- Intuitive (focus on hard examples)
- Practical guidance

**What it explains:**

- How to code AdaBoost
- Why mistakes get more attention
- How to combine weak learners

### View 2: Optimization Perspective

**What it emphasizes:** Loss minimization

**Key idea:** "Minimize exponential loss through coordinate descent"

**Framework:**

- Objective: $\min_F L(F) = \left(\frac{1}{m}\right) \sum_i \exp(-y_i F(x_i))$
- Method: Coordinate descent (optimize one $\alpha_t h_t$ at a time)
- Step size: $\alpha_t = \left(\frac{1}{2}\right) \ln\left(\frac{1-\varepsilon_t}{\varepsilon_t}\right)$ (optimal)

**Strengths:**

- Principled (derived from calculus)
- Convergence guarantees
- Generalizes to other losses

**What it explains:**

- Why the formulas work
- Exponential convergence rate
- Connection to gradient boosting

### View 3: Margin Perspective

**What it emphasizes:** Classification confidence

**Key idea:** "Maximize margins to improve generalization"

**Framework:**

- Margin: $\text{margin}(x, y) = y \frac{F(x)}{\|\alpha\|_1}$
- Goal: Increase margins on all examples
- Result: Better generalization bounds

**Strengths:**

- Explains generalization
- Justifies post-zero training
- Connects to PAC theory

**What it explains:**

- Why test error improves after training = 0
- How margins relate to generalization
- Comparison with SVMs

## The Fragmentation Crisis

⚠️ **Crisis**

**The Three-View Confusion**

Each perspective seems to describe a different algorithm:

- Algorithmic: Heuristic reweighting strategy
- Optimization: Principled loss minimization
- Margin: Generalization-focused design

**Questions:**

1. Are these three different algorithms that happen to be similar?
2. Is one view "correct" and others just interpretations?
3. Do they make different predictions?
4. Which should I use to understand AdaBoost?

Without unification: confusion, fragmented understanding, can't see the big picture!

### The Fragmentation Problem

| Perspective | Domain | What It Seems to Prioritize |
|---|---|---|
| Algorithmic | Implementation | Efficient computation and reweighting |
| Optimization | Theory | Provable convergence and optimality |
| Margin | Generalization | Test error and robustness |

**Apparent conflict:**

- Algorithm view: "It's about focusing on errors"
- Optimization view: "It's about minimizing loss"
- Margin view: "It's about maximizing confidence"

**Which is the "real" AdaBoost?**

## The Surprising Discovery

💡 **Aha Moment**

**The Unification: They're All the Same!**

The three views don't describe different algorithms—they're three perspectives on the **identical** algorithm!

**Key equivalences:**

1. Reweighting = Following the gradient of exponential loss
2. Exponential loss minimization = Implicit margin maximization
3. $\alpha_t$ formula = Optimal step size = Margin-weighted vote

**Every line of the algorithm has three valid interpretations!**

### The Unified Framework

All three views describe this single update:

$$D_{t+1}(i) = \frac{D_{t(i)} \exp(-\alpha_t y_i h_t(x_i))}{Z_t} \tag{59}$$

**Algorithmic interpretation:**

- "Increase weight on mistakes, decrease on correct"

**Optimization interpretation:**

- "Gradient of exponential loss at current point"

**Margin interpretation:**

- "Focus on small-margin examples"

**Same formula, three meanings!**

## Proving the Equivalences

Let's show how each view leads to the same algorithm.

### Equivalence 1: Reweighting = Gradient

**Claim:** AdaBoost's reweighting follows the gradient of exponential loss.

**Proof:**

At round $t$, the exponential loss is:

$$L(F) = \frac{1}{m} \sum_{i=1}^{m} \exp(-y_i F(x_i)) \tag{60}$$

Define $w_i = \exp(-y_i F_{t-1}(x_i))$. The gradient with respect to adding $\alpha h$ is:

$$\frac{\partial L}{\partial \alpha}|_{\alpha=0} = -\frac{1}{m} \sum_{i=1}^{m} y_i h(x_i) w_i \tag{61}$$

The distribution $D_t$ is exactly:

$$D_{t(i)} = \frac{w_i}{\sum_j w_j} = \frac{\exp(-y_i F_{t-1}(x_i))}{Z_t} \tag{62}$$

**Therefore:** Reweighting by $D_t$ = weighting by gradient of exponential loss!

### Equivalence 2: Loss Minimization = Margin Maximization

**Claim:** Minimizing exponential loss implicitly maximizes margins.

**Intuition:**

Exponential loss for example $i$:

$$\ell_i = \exp(-y_i F(x_i)) = \exp(-\text{margin}_i \cdot \|\alpha\|_1) \tag{63}$$

- Large margin → small loss
- Small margin → large loss

**Behavior:**

- Example with margin 0.1: loss = $\exp(-0.1 \cdot \|\alpha\|_1) \approx 0.9$ (high)
- Example with margin 1.0: loss = $\exp(-1.0 \cdot \|\alpha\|_1) \approx 0.37$ (low)

Minimizing exponential loss = minimizing $\exp(-\text{margin})$ = maximizing margin!

### Equivalence 3: Three Interpretations of $\alpha_t$

The weight formula $\alpha_t = \left(\frac{1}{2}\right) \ln\left(\frac{1-\varepsilon_t}{\varepsilon_t}\right)$ has three meanings:

**Algorithmic interpretation:**

- "Weight good hypotheses more (small $\varepsilon_t$ → large $\alpha_t$)"
- Ensures better hypotheses have more influence

**Optimization interpretation:**

- "Optimal step size minimizing $L(F_{t-1} + \alpha h_t)$"
- Derived from setting $\partial \frac{L}{\partial} \alpha = 0$

**Margin interpretation:**

- "Vote weight proportional to edge $\gamma_t = \frac{1}{2} - \varepsilon_t$"
- Better edge → larger margin contribution

**All three describe the same formula from different angles!**

## The Unified Algorithm

Now we can write AdaBoost with all three interpretations:

**Initialize:** $F_0(x) = 0$, $D_1(i) = \frac{1}{m}$

**For** $t = 1, ..., T$:

1. **Train weak learner**
   - Algorithmic: "Train on weighted distribution $D_t$"
   - Optimization: "Choose direction of steepest descent"
   - Margin: "Find hypothesis reducing small-margin examples"
2. **Compute $\boldsymbol{\alpha_t = \left(\frac{1}{2}\right) \ln\left(\frac{1-\varepsilon_t}{\varepsilon_t}\right)}$**
   - Algorithmic: "Weight hypothesis by performance"
   - Optimization: "Optimal step size"
   - Margin: "Vote weight proportional to edge"
3. **Update $\boldsymbol{F_t = F_{t-1} + \alpha_t h_t}$**
   - Algorithmic: "Add weighted hypothesis"
   - Optimization: "Take coordinate descent step"
   - Margin: "Increase ensemble confidence"
4. **Update $\boldsymbol{D_{t+1}(i) \propto D_{t(i)} \exp(-\alpha_t y_i h_t(x_i))}$**
   - Algorithmic: "Reweight based on errors"
   - Optimization: "Follow gradient direction"

- Margin: "Focus on small-margin examples"

**Same algorithm, three complete descriptions!**

## Why This Unification Matters

The unification provides complementary insights:

🔄 **Comparison**

**Question: Why does AdaBoost work?**

**Algorithmic answer:** "It forces weak learners to focus on complementary patterns by reweighting"

**Optimization answer:** "It performs coordinate descent on a convex loss, guaranteeing convergence"

**Margin answer:** "It maximizes classification margins, improving generalization bounds"

**Unified answer:** All three! The reweighting IS the gradient descent IS the margin maximization. They're inseparable aspects of one algorithm.

### Benefits of Each View

| View | Best for Understanding | Key Insight |
|---|---|---|
| Algorithmic | Implementation, intuition | How to code it, why errors matter |
| Optimization | Convergence, theory | Why it works, guarantees |
| Margin | Generalization, when to stop | Why test improves after training=0 |

**Use all three!** Each illuminates different aspects.

## Comparison Table: The Same Formula, Three Ways

The core update $D_{t+1}(i) \propto D_{t(i)} \exp(-\alpha_t y_i h_t(x_i))$:

| Component | Algorithmic | Optimization | Margin |
|---|---|---|---|
| $D_{t(i)}$ | Current weight | Gradient weight | Example importance |
| $y_i h_t(x_i)$ | Correctness | Loss derivative | Signed margin contribution |
| $\exp(-\alpha_t \cdot)$ | Exponential reweight | Natural exponential | Margin-sensitive scaling |
| $\alpha_t$ | Hypothesis strength | Step size | Edge-based vote weight |
| Result | Next weight | Next gradient point | Focus on small margins |

**Every piece has three consistent interpretations!**

## Case Study: One Round, Three Explanations

Let's trace a single boosting round through all three lenses:

**Setup:**

- Round $t = 5$
- Current ensemble: $F_4(x)$
- Train weak learner, get $h_5$ with $\varepsilon_5 = 0.3$

**Compute $\alpha_5$:**

**Algorithmic:** "Hypothesis got 70% weighted accuracy, deserves weight $\alpha_5 = 0.42$"

**Optimization:** "Optimal step along direction $h_5$ is $\alpha_5 = \left(\frac{1}{2}\right)\ln\left(\frac{0.7}{0.3}\right) = 0.42$"

**Margin:** "Edge is $\gamma_5 = 0.2$, vote weight $\alpha_5 = 0.42$ contributes to margins"

**Update weights for example $i$ with $y_i = +1, h_5(x_i) = +1$ (correct):**

**Algorithmic:** "Correctly classified, reduce weight by factor $\exp(-0.42) \approx 0.66$"

**Optimization:** "Gradient component negative here, move weight down by $\exp(-0.42)$"

**Margin:** "Margin increased by 0.42, can reduce focus by factor 0.66"

**Same computation, three valid stories!**

## Connecting to Other Algorithms

The unification reveals connections:

**AdaBoost ↔ Gradient Boosting:**

- AdaBoost: Coordinate descent on exponential loss
- GradientBoost: Coordinate descent on arbitrary differentiable loss
- **Connection:** AdaBoost is a special case!

**AdaBoost ↔ SVMs:**

- Both maximize margins
- SVM: Hard margin (maximize minimum)
- AdaBoost: Soft margin (minimize exponential loss)
- **Connection:** Different margin-based approaches

**AdaBoost ↔ Logistic Regression:**

- Both use convex surrogates for 0-1 loss
- Logistic: Minimize logistic loss
- AdaBoost: Minimize exponential loss
- **Connection:** Different loss functions, same principle

## Practical Implications

**Key Takeaway**

**Using the Unified View**

1. **Choose perspective for the task**
   - Implementing: Use algorithmic view
   - Debugging convergence: Use optimization view
   - Tuning stopping: Use margin view
2. **They predict the same behavior**
   - Algorithmic: "Focus shifts to hard examples"
   - Optimization: "Loss decreases exponentially"
   - Margin: "Confidence increases on all examples"
   - All true simultaneously!
3. **Troubleshooting uses all views**
   - Training error not decreasing? Check edge $\gamma$ (optimization)
   - Test error plateauing? Check margins (margin)
   - Implementation bug? Check reweighting (algorithmic)
4. **This extends to variants**
   - LogitBoost: Same framework, logistic loss
   - L2Boost: Same framework, squared loss
   - Understanding one view helps understand others
5. **Teaching/explaining AdaBoost**
   - Start with algorithmic (intuitive)
   - Add optimization (rigor)
   - Finish with margins (generalization)
   - Show they're unified!

## Summary: From Fragmentation to Unity

🔄 **Comparison**

**Before Unification:**

- "Three different explanations of AdaBoost"
- "Algorithmic view for coding, optimization for theory, margins for generalization"
- "They seem to prioritize different things"
- "Which one is correct?"

**After Unification:**

- "One algorithm, three complementary perspectives"
- "Every component has three valid interpretations"
- "They're inseparable—reweighting IS gradient IS margin focus"
- "All three are correct simultaneously!"

**The Transformation:**

- From: Fragmented understanding with competing views
- To: Unified understanding with complementary lenses

**The Key:**

- Algorithmic + Optimization + Margin = Complete picture
- Reweighting = Gradient of exp loss = Margin maximization
- $\alpha_t$ = Hypothesis weight = Step size = Edge-based vote

## The Big Picture

AdaBoost is remarkable because it simultaneously:

- **Implements** an intuitive reweighting algorithm (easy to code)
- **Optimizes** a principled convex objective (proven guarantees)
- **Maximizes** classification margins (strong generalization)

These aren't three separate properties—they're three descriptions of the identical mathematical object. Understanding this unity provides the deepest insight into why AdaBoost works so well: it's an algorithm where implementation simplicity, optimization theory, and generalization guarantees all align perfectly.

The unification also explains why AdaBoost spawned so many variants (LogitBoost, GentleBoost, etc.) and generalizations (Gradient Boosting): the coordinate descent framework extends to any differentiable loss, carrying forward the same three-way unity.

This completes our journey through boosting theory. We've seen:

1. How weak learners become strong (exponential combination)
2. Where the algorithm comes from (optimization)
3. Why training error vanishes (exponential convergence)
4. Why test error improves afterward (margins)
5. How all perspectives unify (coordinate descent = reweighting = margin maximization)

AdaBoost is not just a practical algorithm—it's a beautiful example of how simple ideas (combine weak learners) can be understood at multiple levels (algorithmic, optimization, statistical), all revealing the same elegant mathematical structure.

# Conclusion: The Complete Picture

We've journeyed through AdaBoost from algorithm to convergence theory:

## The Three Core Insights

1. **Weak Learning Possibility**
   - Crisis: 51% accuracy seems useless
   - Resolution: Adaptive reweighting transforms 51% → 99.9%+
   - Mechanism: Complementary specialists through distribution shifts
2. **Principled Optimization Framework**
   - Crisis: Arbitrary reweighting rules have no guarantees
   - Resolution: Frame as coordinate descent on exponential loss
   - Mechanism: Calculus derives optimal α_t and D_t+1 automatically
3. **Exponential Convergence**
   - Crisis: Expected polynomial, got exponential $\exp(-2\gamma^2 T)$
   - Resolution: Convexity enables multiplicative progress
   - Mechanism: Each round multiplies error by constant $< 1$

## The Unifying Thread

All insights connect through: **AdaBoost performs coordinate descent on exponential loss**, which:

- Enables exponential convergence (convexity)
- Derives reweighting automatically (optimization)
- Combines weak learners optimally (greedy coordinate selection)

## Practical Implications

🎯 **Key Takeaway**

**Using AdaBoost Effectively**

1. **Weak learners**: Any $\gamma > 0$ works; larger $\gamma$ gives faster convergence
2. **Number of rounds**: Use $T \approx \ln(1/\varepsilon)/(2\gamma^2)$; typically 100-500 rounds
3. **Formula is optimal**: Don't modify α_t or D_t updates
4. **Simple base learners**: Decision stumps work excellently

## Next Week: Generalization Theory

Week 6 answered: **How does AdaBoost work and why does training error converge?**

Week 7 will answer: **Why does AdaBoost generalize beyond training data?**

- The zero training error paradox (test error improves after training = 0)
- Margin theory and generalization bounds
- Loss function analysis and comparison
- Practical guidelines for boosting